



**FACULTAD CIENCIAS Y TECNOLOGIA
RECINTO TOMAS BORGE**

**Carrera: INGENIERÍA EN TECNOLOGÍA DE LA INFORMACIÓN CON
ÉNFASIS EN DESARROLLO WEB Y MARKETING DIGITAL.**

Trabajo No: 3.

**CineNow: Plataforma Full-Stack para la Administración Cinematográfica con
Arquitectura RBAC y Persistencia Relacional**

Año cursado: 5to.

Grupo: 1.

**Componente: Electiva PPF II: Portafolio Proyecto App Para Web En
Inglés/Español**

Integrantes:

Carnet:

- | | |
|------------------------------------|-----------------|
| ➤ Brayan Exael Ortiz Montes. | 22 - 03387 - 1. |
| ➤ Norvin Lloeldi Gutiérrez Zapata. | 22 - 02481 - 1. |
| ➤ David Josué Sandoval Marengo. | 22 - 03749 - 1. |

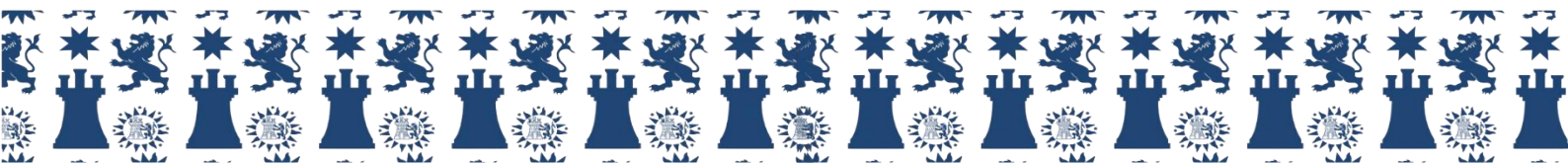
Docente.: Aldo berrios

León, Nicaragua.

24 de septiembre del 2026.

www.unanleon.edu.ni

2025: 49/19 ¡Siempre más allá, Seguimos Avanzando en la Revolución!



INFORME TÉCNICO DE PROYECTO

1. INTRODUCCIÓN Y OBJETIVOS

1.1. Introducción

En el ámbito del desarrollo web moderno, la separación de responsabilidades entre la capa de presentación (Frontend) y la capa de lógica de negocio (Backend) se ha consolidado como un estándar de la industria. **CineNow** nace como una solución Full-Stack diseñada para gestionar de manera centralizada y segura un catálogo de obras cinematográficas. La plataforma ofrece control de acceso dinámico, gestión reactiva en tiempo real y persistencia relacional estricta bajo una interfaz gráfica optimizada en modo oscuro.

1.2. Objetivos del Proyecto

- **Objetivo General:** Desarrollar una aplicación web monolítica desacoplada para la administración de un catálogo cinematográfico con control de acceso basado en roles (RBAC) y filtrado dinámico de información.
- **Objetivos Específicos:**
 1. Implementar un backend sólido sobre **Laravel 13** utilizando API RESTful protegida por autenticación con tokens HTTP mediante **Laravel Sanctum**.
 2. Abstraer el acceso a la base de datos MySQL aplicando el patrón **Eloquent ORM** para establecer relaciones 1: *N* entre las entidades de Géneros y Películas.
 3. Desplegar un frontend interactivo utilizando **React 19** y **Vite**, garantizando filtrados en tiempo real e integración transparente con la API.
 4. Garantizar la seguridad del sistema mediante un esquema RBAC utilizando **Spatie Laravel-Permission** y un flujo estricto de aprobación de cuentas para nuevos usuarios.

2. ARQUITECTURA DE SOFTWARE Y STACK TECNOLÓGICO

El sistema CineNow adopta una arquitectura de software desacoplada de tipo cliente-servidor, donde el frontend actúa como una Single Page Application (SPA) que consume los servicios expuestos por la API RESTful del backend.



2.1. Capa Backend (backend/)

- **Laravel 13 (PHP 8.2+):** Elegido por su robustez, sistema de migraciones, ORM avanzado y elegancia sintáctica para la construcción de APIs.
- **Laravel Sanctum:** Utilizado para la emisión y verificación de *Bearer Tokens*, garantizando el aislamiento de sesiones entre clientes.
- **Spatie Laravel-Permission:** Paquete integrado para la gestión de roles (Administrador, Consulta, etc.) y permisos individuales vinculados a las rutas de la API.

2.2. Capa Frontend (frontend/)

- **React 19 + Vite:** Permite una renderización ultra rápida, manejo reactivo del estado de la interfaz y tiempo de construcción compilado mínimo.
- **React Router Dom:** Encargado del enrutamiento del lado del cliente sin recargar el navegador.
- **Context API (AuthContext):** Almacena de forma centralizada la información del usuario autenticado, tokens y un helper global `hasPermission()` para controlar la visibilidad de componentes UI.
- **CSS3 Custom (Tema Oscuro):** Interfaz construida con variables CSS, acentos en rojo e intensidad oscura pensada para experiencias de streaming/cine.

3. MODELO DE BASE DE DATOS Y PERSISTENCIA ELOQUENT ORM

El modelo relacional fue optimizado para cumplir con las normas de normalización y abstraer la lógica de relaciones mediante Eloquent.

3.1. Diccionario de Datos (Entidades Clave)

Tabla: generos

Campo	Tipo de Dato	Clave / Restricción	Descripción
id	BigInteger (Auto)	Primary Key	Identificador único del género
nombre	VARCHAR(255)	Unique, Not Null	Nombre del género (Acción, Terror, etc.)
timestamps	Timestamp	Nullable	Fechas created_at y updated_at

Tabla: peliculas

Campo	Tipo de Dato	Clave / Restricción	Descripción
id	BigInteger (Auto)	Primary Key	Identificador único de la película
titulo	VARCHAR(255)	Not Null	Título comercial
sinopsis	TEXT	Nullable	Descripción o resumen del argumento
genero_id	Foreign ID	Foreign Key (generos.id)	Clave foránea que relaciona la película con su género
duracion	Integer	Not Null	Duración total expresada en minutos
clasificacion	VARCHAR(50)	Not Null	Clasificación por edad (PG-13, R, etc.)
activo	Boolean	Default (True)	Estado de visibilidad dentro del catálogo
timestamps	Timestamp	Nullable	Fechas de creación y actualización

3.2. Implementación de Relaciones Eloquent ORM (1: N)

1. Modelo Genero.php:

```
cinenow-react-laravel > backend > app > Models > Genero.php
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6  use Illuminate\Database\Eloquent\Relations\HasMany;
7
8  class Genero extends Model
9  {
10     protected $table = "generos";
11     protected $fillable = [
12         'nombre',
13     ];
14
15     /**
16      * Relationship: Genero has many Peliculas.
17      */
18     public function peliculas(): HasMany
19     {
20         return $this->hasMany(Pelicula::class);
21     }
22 }
```

2. Modelo Pelicula.php:

```
cinenow-react-laravel > backend > app > Models > Pelicula.php
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6  use Illuminate\Database\Eloquent\Relations\BelongsTo;
7
8  class Pelicula extends Model
9  {
10     protected $fillable = [
11         'titulo',
12         'sinopsis',
13         'genero_id',
14         'duracion',
15         'clasificacion',
16         'activo',
17     ];
18
19     /**
20      * Relationship: Pelicula belongs to Genero.
21      */
22     public function genero(): BelongsTo
23     {
24         return $this->belongsTo(Genero::class);
25     }
26 }
```

3. Resolución Transparente en PeliculaController.php:

- **Carga Ansiosa (*Eager Loading*):** En el método index(), se ejecuta `Pelicula::with('genero')->get()` evitando el problema N+1 de consultas en SQL.
- **Inyección Transparente:** En los métodos store() y update(), el controlador recibe la cadena de texto del género enviada desde el frontend y ejecuta `Genero::firstOrCreate(['nombre' => $request->genero])`. Esto permite vincular la llave foránea `genero_id` sin requerir modificaciones en la interfaz de usuario existente.

4. SEGURIDAD, AUTENTICACIÓN Y ROLES (RBAC)

4.1. Flujo de Aprobación de Cuentas

1. Un nuevo usuario completa el formulario de registro en la vista Auth.jsx.
2. El backend registra la cuenta pero establece su estado de aprobación como **pendiente**.
3. Al intentar iniciar sesión, el frontend redirige al usuario a la vista restringida PendingApproval.jsx hasta que un usuario con rol de **Administrador** apruebe su acceso desde el panel Usuarios.jsx.

4.2. Control de Acceso basado en Roles

- **Middleware de Protección:** Las rutas críticas en api.php utilizan el middleware `permission: peliculas.crear`, `permission: peliculas.editar`, etc.
- **Seguridad Reactiva en Frontend:** El helper `hasPermission()` analiza los permisos incluidos en el token del usuario y oculta o muestra dinámicamente los botones de *Agregar película*, *Editar* y *Eliminar*.

5. COMPONENTES Y FUNCIONALIDADES DESTACADAS EN FRONTEND

5.1. Catalogo de Películas y Buscador Reactivo (*Peliculas.jsx*)

- **Búsqueda en Tiempo Real:** Implementación de un estado local (busqueda) ligado a una variable derivada (peliculasFiltradas) que evalúa en tiempo real si el título de la película contiene el patrón ingresado, omitiendo diferencias entre mayúsculas y minúsculas.
- **Renderizado Adaptativo:** La interfaz evalúa si el género retornado por la API viene estructurado como un objeto relacional (pelicula.genero.nombre) o como cadena simple, garantizando cero errores de renderizado.
- **Paneles Métrica:** Muestra indicadores en tiempo real como el total de películas registradas, películas activas y estado de conexión con el backend.

6. CONCLUSIONES

El desarrollo de la plataforma **CineNow** demostró ser una solución Full-Stack sólida y escalable. Se lograron integrar con éxito las capacidades avanzadas de **Laravel 13** en el manejo relacional de bases de datos y seguridad API, con la velocidad de respuesta y dinamismo que ofrece **React 19** en el cliente. La implementación de la relación Eloquent 1:*N* y del filtro en tiempo real enriquecieron la experiencia general del usuario manteniendo una estricta rigurosidad técnica.